



Deep Graph Clustering Using Graph Neural Networks and Seed Detection

Carlos Pereira Lopes Filho¹(✉) , Guilherme Henrique Messias¹ ,
Sylvia Iasulaitis² , Alneu de Andrade Lopes³ ,
and Alan Demétrius Baria Valejo¹

¹ Department of Computing, Federal University of São Carlos, São Carlos, Brazil
`carlos.lopes@draiven.io`, `ghmessias@estudante.ufscar.br`,
`alanvalejo@ufscar.br`

² Department of Social Sciences, Federal University of São Carlos, São Carlos, Brazil
`si@ufscar.br`

³ University of São Paulo, São Carlos, Brazil
`alneu@icmc.usp.br`

Abstract. Clustering plays a fundamental role in attributed graphs, which incorporate both topological structure and node attributes represented as feature vectors. Deep clustering methods based on Graph Neural Networks (GNNs) have proven effective in extracting patterns from such data. Most existing approaches use a traditional clustering algorithm to identify representative elements, which are later employed to train a GNN and, finally, the clustering task. However, when selecting representative elements, these clustering algorithms consider only the feature vector of each instance, neglecting topological information. This limitation negatively impacts the GNN learning process. To address this issue, we propose Deep Graph Clustering via Graph Neural Network and Seed Detection (DGCSD), a model consisting of three modules: (1) the seed detection module, which identifies representative nodes; (2) the embedding module, which employs a graph attentional network to capture global topological information; and (3) the self-supervised module, which leverages the representative nodes to guide the clustering task. An advantage of our algorithm is that it integrates both information, the topological structure and node attributes across all modules to identify representative elements. This is the first GNN-based clustering algorithm that incorporates seed detection, establishing a significant reference for future research. The empirical analysis of real-world graphs provides evidence that combining a seed detection algorithm with a GNN model is competitive compared to well-established algorithms.

Keywords: Graph Neural Networks · Seed Detection · Graph Clustering · Community Detection · Node Clustering · Clustering

1 Introduction

Graph representation data arise in various domains, including social networks, citation networks, biological networks, and traffic flow systems [21]. Currently,

there is growing interest in modeling real systems by using a graph representation in which, beyond the graph topological structure, the nodes also have a feature vector associated to each of them. Such graphs can be analyzed using machine learning algorithms for node classification or community detection. In recent years, Graph Neural Networks (GNNs) have gained prominence in the scientific community [26] due to their capacity to handle diverse learning paradigms. While GNNs have traditionally been employed for semi-supervised node classification, they are increasingly being used for clustering as well, aiming to group nodes that are similar to each other and dissimilar to nodes in other groups [25].

GNN-based clustering methods are recent and can be broadly categorized into *Graph Embedding Models* and *Deep Models* [14]. Embedding-based methods typically follow a two-step process. In the first, *Graph Embedding Models*, a Graph Neural Network (GNN) is used to learn latent node representations—commonly known as node embeddings—that capture both structural and attribute information. Second, these embeddings serve as input to an external clustering algorithm, such as K-Means. While effective, it separates the representation learning from the clustering process, meaning that the embeddings are learned without directly considering the clustering objective. As a limitation, the model cannot refine the embeddings to improve clustering quality, limiting the potential of truly end-to-end solutions [9, 28].

In contrast, *Deep Models* approaches aim to incorporate a loss function that directly optimizes the clustering process. For instance, [15] optimizes the modularity, a well-known graph quality function. However, as discussed in [5], modularity-based measures fail to detect communities of a certain size. Furthermore, defining an effective loss function for this purpose remains a significant challenge. A common solution is to initially employ an external clustering algorithm to identify a set of representative nodes, which are then used to train a GNN, like the DNENC (Deep neighbor-aware embedding for node clustering) algorithm, proposed in [23]. This strategy eliminates the need for a complex loss function, as the selected representative nodes naturally guide the GNN model toward group formation. Specifically, these nodes act as centroids, assisting the GNN model in differentiating between clusters and organizing similar nodes accordingly. However, a limitation of this strategy is that the external clustering algorithm considers only attribute information to select representative nodes, neglecting topological information.

To overcome these limitations, this study introduces **DGCS** (*Deep Graph Clustering with Seed Detection*), a framework comprising three key modules. The first module, the **Seed Detection Module**, identifies representative nodes using a seed detection algorithm that leverages the graph's topological structure, incorporating metrics such as Betweenness Centrality (BC), Weighted Betweenness Centrality (WBC), Closeness Centrality (CC), and Random Initialization. The second module, the **Embedding Module**, employs a Graph Autoencoder (GAE) [12] with an attentional encoder that effectively captures relationships among central nodes [27], while an Inner Product Decoder reconstructs the graph structure. Finally, the **Semi-Supervision Module** integrates

a Kullback-Leibler (KL) divergence function to quantify the discrepancy between the predicted and target distributions, optimizing it jointly with the reconstruction loss. This component enhances the learned embeddings, making them more suitable for clustering. The main contributions of this work are summarized as follows:

- **A formal integration of seed detection strategies with Graph Neural Networks (GNNs):** A unified approach that simultaneously leverages attributed and topological information to enhance node clustering. This integration improves the learning capacity of GNNs in unsupervised settings by guiding the model toward more informative regions of the graph. The proposed formulation offers a theoretical basis to support future advances in graph-based representation learning. This integration is flexible and can be adapted to other unsupervised tasks, such as link prediction and graph-based anomaly detection.
- **A novel GNN-based model named DGCS (Deep Graph Clustering with Seed Detection):** The model combines seed detection strategies with attention mechanisms to guide the learning process, promoting the selection of structurally relevant nodes. The architecture incorporates a Kullback-Leibler (KL) divergence-based loss function, enabling the Graph Attention Network (GAT) to generate discriminative and cluster-aware embeddings. This design encourages the model to produce latent representations that reflect both local and global structural patterns of the graph.
- **An open-source framework for graph clustering:** We release a fully-documented and extensible toolkit that implements the proposed methodology. The framework is designed to support reproducibility, ease of experimentation, and further research. It allows users to adapt or extend the seed detection strategies, GNN architectures, and evaluation pipelines, making it a valuable resource for the graph machine learning community.

We conducted an empirical evaluation on three commonly used GNN benchmark datasets, comparing their performance with state-of-the-art algorithms (traditional and GNN-based algorithms). Our analysis included a visual inspection of the clustering results and the behavior of the algorithm’s loss function. The results provide compelling evidence that the proposed algorithm exhibits competitive performance when compared to algorithms in the field.

2 Related Work

We briefly review prior work on GNN-based clustering methods, organizing them into two categories: *Graph Embedding Models*, which focus on learning node representations by leveraging both node features and the graph structure; and *Deep Models*, relies on deep neural networks to jointly learn latent representations and optimize clustering objectives.

2.1 Graph Embedding Models

Graph embedding models map nodes into a latent space, typically via Graph Autoencoders (GAEs), and then apply vector-space clustering (e.g., K-Means or spectral clustering) to detect communities [2]. To alleviate over-squashing, algorithms modify graph topology: DGEN uses neighborhood pooling [24], personalized PageRank reweights edges [28], and multigraphs capture diverse local views [20]. Adversarial approaches like ARGA [16] and capsule-based GANs [10] [19] enforce distributional alignment and improve handling of isolated nodes. Recent refinements—linear latent models [18], Laplacian-filtered MLPs (GLASS) [22], and attention-based GATs with centrality encoding [27], enhance scalability and combat over-smoothing. However, decoupling embedding and clustering can limit performance, motivating integrated deep clustering architectures.

2.2 Deep Models

Deep models can follow two main strategies: (1) directly optimizing a cluster quality measure or (2) identifying representative nodes to guide the clustering process. The first aims to optimize an objective function that accurately captures the concept of clustering. Since clustering is inherently unsupervised, defining robust objective functions remains a key challenge [8, 14]. In the latter approach, an external clustering algorithm is employed to provide preliminary information, which serves as initial guidance for subsequent GNN training. This training process then refines existing groups or uncovers more nuanced partitions [1]. Integrating external clustering with a GNN enables the model to learn richer and more complex representations by combining node features with the graph’s topological structure, leveraging information that may not be immediately evident from the original graph alone.

One emerging approach in deep graph clustering tasks is the integration of traditional clustering algorithms with Graph Neural Networks (GNNs). A recent algorithm is the Deep Neighbor-aware Embedded Node Clustering (DNENC) framework [23]. In DNENC, a deep graph autoencoder is used to learn latent node representations by leveraging both node attributes and the graph structure through either a graph convolutional or attentional encoder. Traditional clustering, specifically K-Means, is then applied to these embeddings to generate soft cluster assignments that guide a self-training module. This module jointly refines both the learned embeddings and the clustering results in an end-to-end manner. DNENC, therefore, exemplifies how established clustering techniques can be combined with modern deep learning architectures to enhance performance in graph clustering tasks.

UCoDe [15] leverages GNNs to compute modularity in a more resource-efficient manner. Its loss function seeks to maximize intra-community connections while minimizing inter-community interactions. UDGc [9] employs a mutual optimization strategy within an end-to-end framework to form coherent clusters. [4] introduces a variational graph autoencoder specifically designed for community detection. EG-VGAE [3] uses a variational graph autoencoder

designed for community detection, which integrates evolution information to overcome the limitations of static networks.

Alternatively, many deep clustering approaches rely on shaping their cost functions around a target distribution. This distribution is informed by representative nodes for each cluster, typically identified via conventional clustering algorithms such as K-Means. These traditional algorithms offer straightforward implementation, strong clustering performance, and effectively pinpoint representative examples [8]. Furthermore, imposing an auxiliary distribution encourages the natural formation of well-separated clusters by guiding nodes toward cohesive groupings in the latent space [14].

3 Deep Graph Clustering with Seed Detection

We introduce **DGCSD** (*Deep Graph Clustering with Seed Detection*), a **novel** Graph Neural Network (GNN) approach for node clustering. The key innovation of DGCSD lies in its two-stage procedure. Instead of relying on traditional external clustering algorithms, such as K-Means or community detection algorithms, to provide preliminary structural cues, our approach leverages efficient *seed detection* mechanisms to detect potential clusters’ centroids (well-known as “seeds”). These seeds serve as reference points that capture essential aspects of the graph’s topology and guide the subsequent GNN-based embedding process. The DGCSD code can be found on GitHub (<https://github.com/carlosplf/DGCSS>).

Specifically, our algorithm begins by identifying potential seeds using one of three distinct mechanisms: Betweenness Centrality, Closeness Centrality, or Random Seeds. By providing these seeds as initial structural cues, the GNN can focus on refining the groupings and uncovering more nuanced cluster boundaries. This integration of seed detection directly into the clustering pipeline reduces the dependence on large-scale external clustering algorithms and has the potential to improve its efficiency and performance.

Once the seeds are selected, the second stage of DGCSD uses a Graph Autoencoder (GAE) to embed the nodes into a lower-dimensional latent space, preserving both local and global structural information for more effective clustering. After obtaining these preliminary embeddings, a dedicated *Self-Supervised Clustering* component refines them by combining the GAE’s reconstruction loss with a clustering loss. The clustering loss focuses on pulling nodes closer to their assigned centroids. By jointly optimizing both the GAE and the clustering module, the model balances structural reconstruction with cluster cohesion, arriving at more robust, well-separated representations in the latent space.

In summary, *DGCSD* is implemented in three main steps, called modules: **Seed Detection**; **Embedding via GAE**; **Self-Supervised Clustering**. This end-to-end strategy first anchors centroids within the graph, then learns embeddings guided by these centroids, and finally refines clusters through self-supervision, resulting in a principled and effective new approach to graph-based clustering. Figure 1 illustrates the full architecture.

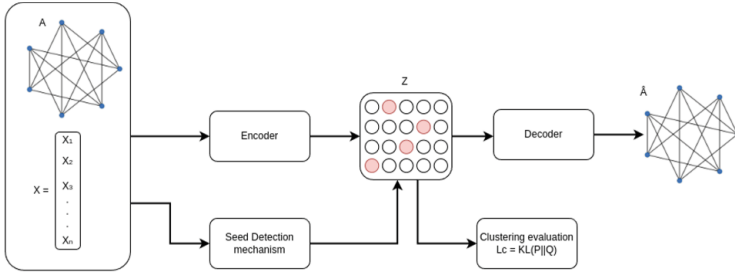


Fig. 1. DGCSA Architecture: The input consists of a graph defined by the adjacency matrix A and node attribute vector X . A seed detection mechanism is applied based on the graph topology, selecting centroid nodes within the embedding matrix Z . In addition to the standard Graph Autoencoder (GAE) loss, a clustering loss is computed and integrated into the overall graph loss.

3.1 Seed Detection Module

Seed detection in graph-based clustering involves identifying a set of initial representative nodes (often referred to as centroids or “seeds” or clusters’ centroids) [11] that serve as a starting point for the clustering process. By detecting these seeds, the algorithm can leverage them to guide the learning process, helping it converge to cohesive substructures within the network. Various strategies exist to select seeds [7], ranging from random sampling to centrality-based approaches that consider node influence (e.g., Betweenness or Closeness Centrality). The choice of these seeds is crucial because it can significantly impact both the speed of convergence and the quality of the final clusters.

Definition 1 (Basic Definitions). *Formally, we define a graph as an ordered pair $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of nodes and $E \subseteq V \times V$ is the set of edges. Each node $v_i \in V$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$, where d is the dimensionality of the node features. The feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ represents the set of all node feature vectors.*

Definition 2 (Seed Detection). *Let $c : V \rightarrow \mathbb{R}$ denote a centrality measure (e.g., Betweenness or Closeness Centrality) that quantifies the influence of each node. The seed detection problem consists of selecting a subset of nodes $U \subseteq V$, with $|U| = k$, such that:*

$$U^* = \arg \max_{U \subseteq V, |U|=k} \sum_{v \in U} c(v), \quad (1)$$

in some cases, one may also impose additional constraints (e.g., ensuring that the selected nodes are spread across different regions of the graph) to improve diversity and coverage. The nodes in U^ then serve as the representative seeds (or centroids) that guide the clustering process [7, 11].*

As part of DGCSO, we use Seed Detection mechanisms to identify the initial centroids for execution. Based on these initial centroids, the algorithm has a starting point for performing clustering learning, since the loss functions rely on the distances between nodes and centroids to estimate loss values and train the network. For seed detection, we propose the use and comparison of three distinct mechanisms: *Betweenness Centrality* [6], *Closeness Centrality* [17], and *Random Seeds*. Given that each of these algorithms is based on different metrics for selecting centroids, the outcome of these choices directly influences network training and, consequently, the results of the clustering process.

Definition 3 (Betweenness Centrality). For any two distinct nodes $s, t \in V$, denote by σ_{st} the total number of shortest paths between s and t , and by $\sigma_{st}(v)$ the number of those paths that pass through node $v \in V$. The betweenness centrality of node v is defined as:

$$BC(v) = \sum_{\substack{s, t \in V \\ s \neq v \neq t}} \frac{\sigma_{st}(v)}{\sigma_{st}}. \quad (2)$$

This measure quantifies the importance of node v in facilitating communication between other nodes in the graph by capturing the frequency with which v lies on the shortest paths between pairs of nodes.

Definition 4 (Closeness Centrality). For a node $v \in V$, let $d(v, u)$ denote the shortest path distance between v and any other node $u \in V$. The closeness centrality of node v is defined as:

$$CC(v) = \frac{1}{\sum_{u \in V \setminus \{v\}} d(v, u)}. \quad (3)$$

A normalized version is often used to account for differences in graph size:

$$CC_{norm}(v) = \frac{|V| - 1}{\sum_{u \in V \setminus \{v\}} d(v, u)}. \quad (4)$$

This measure quantifies the average closeness of node v to all other nodes in the graph, with higher values indicating that v is, on average, closer to all other nodes.

3.2 Embedding Module

The embedding process is carried out using a Graph Autoencoder (GAE) based on PyTorch, following the principles described in [12]. In this context, our goal is to project the graph nodes into a latent space of reduced dimensionality, while preserving adjacency relationships as much as possible. To achieve this, we employ a neural network that encodes the graph information (adjacency values and possible node attributes) into a latent vector, aiming to capture the inherent structures of the graph and, simultaneously, facilitate subsequent

analysis tasks such as clustering or link prediction. Specifically, the *Encode* step can be described by the following equation:

$$Z = \sigma(\tilde{A}XW), \quad (5)$$

where $\tilde{A}$ represents the normalized adjacency matrix, X is the matrix of node attributes, W are the trainable weights of the model, and $\sigma(\cdot)$ denotes the activation function (e.g., *ReLU*). Thus, Z constitutes the latent representation of each node, which subsequently undergoes the decoding process, where the model attempts to reconstruct the original graph structure or predict relationships between nodes. This approach addresses both the learning of discriminative representations and the generation of plausible structures, aligning with the proposal of [12].

3.3 Self-Supervised Module and Loss

In this stage, we employ a self-supervision module named DGCSO, in which the learned cluster centroids $\{\mu_u\}$ explicitly guide the embedding process by minimizing the clustering loss component, these centroids are iteratively updated to shape the latent space and accelerate convergence.

DGCSO integrates the clustering process directly into the training of the Graph Autoencoder (GAE). While standard GAE frameworks rely solely on the adjacency matrix for computations, DGCSO implements a matrix M [23] that not only considers immediate neighbors but also encompasses t -hop neighbors. This is accomplished via a transition matrix B , where

$$B_{ij} = \frac{1}{d_i},$$

and d_i denotes the degree of node i . Each element B_{ij} thus represents the probability of a random walk step from node i to node j . By employing

$$M = (B + B^2 + \dots + B^t)t,$$

instead of the original adjacency matrix $\tilde{A}$ in the GNN's training procedure, we incorporate the relevance of multi-hop neighbors into the learning process.

Regarding the loss function, DGCSO constructs its objective L by summing two components: the Graph Autoencoder reconstruction error, L_r , and the clustering error, L_c , scaled by a weighting factor γ . Formally, the total loss is given by $L = L_r + \gamma \cdot L_c$. While L_r measures the discrepancy between the original adjacency matrix and its reconstructed counterpart, L_c guides the clustering of nodes in the latent space. For each node z_i , we employ a *t-Student* distribution (Eq. 6) to measure the proximity between z_i and the centroids μ_u of each cluster, resulting in a matrix Q_{iu} :

$$q_{iu} = \frac{(1 + \|z_i - \mu_u\|^2)^{-1}}{\sum_k (1 + \|z_i - \mu_k\|^2)^{-1}}. \quad (6)$$

Based on Q , we calculate a probability distribution P (Eq. 7), representing the likelihood that node i belongs to cluster u :

$$p_{iu} = \frac{\frac{q_{iu}^2}{\sum_i q_{iu}}}{\sum_k \left(\frac{q_{ik}^2}{\sum_i q_{ik}} \right)}. \quad (7)$$

Finally, the clustering error L_c is obtained via the Kullback–Leibler [13] divergence between P and Q :

$$L_c = KL(P \parallel Q) \quad (8)$$

This self-supervision mechanism ensures that the clustering process evolves in tandem with the GAE training, continuously guiding the network to refine its internal representations. By monitoring the clustering distribution via L_c , the model produces latent representations that are not only highly discriminative but also attentive to multi-hop neighborhood structures, ultimately yielding more cohesive clusters and improved predictive performance.

4 Experiments

4.1 Metrics

To assess the quality of the clusters, we employ three commonly used metrics in clustering evaluation: **Normalized Mutual Information (NMI)**, **Adjusted Rand Index (ARI)**, and **Modularity**. NMI measures the amount of shared information between the predicted clusters and the ground truth, normalized to ensure values lie between 0 and 1, with higher scores indicating better alignment. ARI evaluates the similarity between cluster assignments and ground-truth labels while accounting for chance, offering a score that can take negative values in the worst case but typically ranges from 0 to 1. Finally, Modularity quantifies the strength of the division of a graph into clusters, with larger values indicating a more pronounced community structure.

4.2 Datasets

We employ three widely used benchmark datasets—Cora, Citeseer, and Amazon Photo—as provided in the **Planetoid** and **Amazon** classes of PyTorch Geometric¹. The Cora and Citeseer datasets are citation networks where nodes represent scientific publications and edges denote citation relationships. In contrast, the Amazon Photo dataset is a product co-purchasing network in which nodes correspond to products in the photo category and edges indicate co-purchasing relationships. Here, node features are derived from product metadata, and labels correspond to product categories. More details in Table 1.

These datasets are standard benchmarks for evaluating graph-based models, especially GNNs, and come with pre-defined training, validation, and test splits.

¹ https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Amazon.html.

Table 1. Benchmark Graph Datasets.

Dataset	Nodes	Features	Clusters	Links	Content Words
Cora	2,708	1,433	7	5,429	3,880,564
Citeseer	3,327	3,703	6	4,732	12,274,336
Amazon Photo	7,650	745	8	238,162	- - -

4.3 Parameters

In all of our experiments, we employed the following hyperparameters: **P calculation interval:** 10; **Clustering loss gamma:** 20; **Hidden layer size:** 64 for Cora and Citeseer, 128 for Amazon Photo; **Output layer size:** 16; **Epochs:** 400.

These baseline values underpin our experiments. Preliminary tests indicate more hidden neurons can improve clustering—but we maintain 64 for fairness—and the loss coefficient γ critically affects performance. Deeper networks show diminishing returns, whereas learning-rate decay enhances convergence stability.

4.4 Results

We evaluated the performance of our algorithm (DGCSD) and compared it with several clustering algorithms, including traditional algorithms (K-Means, Fast-Greedy, and Spectral Clustering) as well as GNN-based approaches (GAE + K-Means, VGAE + K-Means, and DNENC). To assess the results, we conducted three analyses: (i) a visual inspection of the clusters, (ii) a benchmark comparison with relevant algorithms, and (iii) an in-depth analysis of hyperparameters and loss values.

4.5 Visual Analysis

Figure 2 and 3 show the clustering results obtained by the DGCSD algorithm on the Cora, Citeseer, and Amazon Photo datasets, respectively. These visualizations illustrate the algorithm’s ability to effectively capture and represent the underlying structure of each dataset, resulting in well-separated clusters that closely reflect the true data distributions.

4.6 Benchmark and Comparisons

We compared our algorithm (DGCSD) with both traditional clustering approaches (K-Means, FastGreedy, and Spectral Clustering) and GNN-based algorithms that leverage graph representations, including GAE and VGAE [12] combined with K-Means, as well as DNENC. The objective was to evaluate how effectively each algorithm exploits the graph structure to form meaningful clusters.



Fig. 2. Clustering plots for DGCS D combined with BC and CC on Cora dataset using the t-SNE algorithm for visualization.



Fig. 3. Clustering plots for DGCS D combined with BC and CC on Amazon Photo dataset using the t-SNE algorithm for visualization.

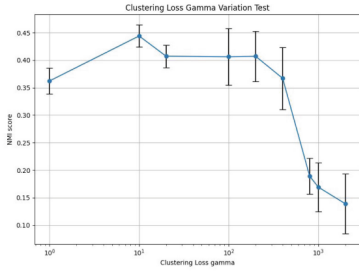
DGCS D demonstrated promising performance, achieving the best results on certain datasets, particularly Cora and Amazon Photo, from NMI and ARI metrics. Although it did not outperform state-of-the-art algorithms across all metrics and datasets, it consistently delivered competitive outcomes. The algorithm successfully identified coherent clusters in both the BC (Betweenness Centrality) and CC (Closeness Centrality) configurations, confirming its capacity to capture the underlying structure of the data.

4.7 Hyperparameters Variation

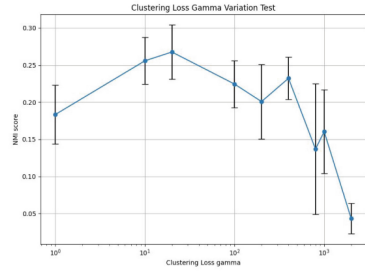
We investigated the impact of varying both the hidden layer size and the clustering loss parameter γ on the performance of the DGCS D variants. These experiments aim to assess how changes in network capacity and the clustering loss influence performance on benchmark datasets. Although increasing the hidden layer size can lead to performance gains, it also introduces a significant increase in execution time, highlighting a trade-off between improved accuracy and computational cost.

Table 2. Clustering performance comparison of different algorithms on the Cora, Citeseer, and Amazon Photo datasets. The reported metrics are Normalized Mutual Information (NMI), Adjusted Rand Index (ARI), and Modularity (Mod.). **DGCSD + CC** and **DGCSD + BC** refer to our algorithm using Closeness and Betweenness Centrality for seed detection, respectively.

Algorithm	Cora			Citeseer			Amazon Photo		
	NMI	ARI	Mod.	NMI	ARI	Mod.	NMI	ARI	Mod.
K-Means	0.077 (± 0.06)	0.026 (± 0.04)	0.128 (± 0.08)	0.154 (± 0.09)	0.107 (± 0.08)	0.245 (± 0.07)	0.112 (± 0.00)	0.045 (± 0.00)	0.060 (± 0.00)
FastGreedy	0.007 (± 0.00)	-0.002 (± 0.00)	0.002 (± 0.00)	0.003 (± 0.00)	0.000 (± 0.00)	0.000 (± 0.00)	0.002 (± 0.00)	0.000 (± 0.00)	0.000 (± 0.00)
SpectralClustering	0.141 (± 0.01)	0.107 (± 0.01)	0.190 (± 0.01)	0.208 (± 0.00)	0.192 (± 0.00)	0.291 (± 0.00)	0.421 (± 0.00)	0.290 (± 0.00)	0.315 (± 0.00)
GAE + K-Means	0.454 (± 0.03)	0.373 (± 0.06)	0.721 (± 0.01)	0.312 (± 0.00)	0.287 (± 0.00)	0.743 (± 0.00)	0.524 (± 0.11)	0.309 (± 0.07)	0.577 (± 0.13)
VGAE + K-Means	0.450 (± 0.02)	0.371 (± 0.05)	0.720 (± 0.01)	0.343 (± 0.07)	0.331 (± 0.07)	0.740 (± 0.07)	0.555 (± 0.03)	0.376 (± 0.05)	0.592 (± 0.05)
DNENC	0.428 (± 0.07)	0.338 (± 0.09)	0.693 (± 0.03)	0.279 (± 0.02)	0.266 (± 0.03)	0.686 (± 0.04)	0.559 (± 0.03)	0.411 (± 0.03)	0.642 (± 0.02)
DGCSD + CC	0.458 (± 0.04)	0.402 (± 0.05)	0.670 (± 0.03)	0.235 (± 0.02)	0.179 (± 0.03)	0.611 (± 0.04)	0.570 (± 0.06)	0.457 (± 0.06)	0.635 (± 0.03)
DGCSD + BC	0.325 (± 0.03)	0.255 (± 0.05)	0.569 (± 0.02)	0.237 (± 0.01)	0.180 (± 0.02)	0.557 (± 0.06)	0.497 (± 0.02)	0.364 (± 0.02)	0.604 (± 0.07)
DGCSD + Random	0.296 (± 0.05)	0.238 (± 0.07)	0.486 (± 0.05)	0.245 (± 0.03)	0.215 (± 0.05)	0.702 (± 0.04)	0.587 (± 0.04)	0.453 (± 0.06)	0.624 (± 0.02)



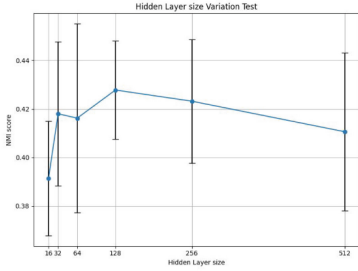
(a) DGCSD + CC on Cora: Clustering Loss Gamma variation.



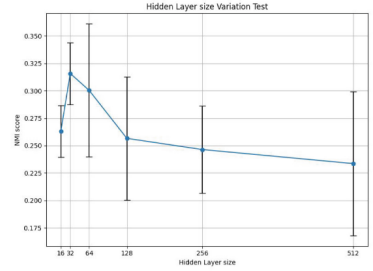
(b) DGCSD + CC on Citeseer: Clustering Loss Gamma variation.

Fig. 4. Variation in the clustering loss parameter γ for the DGCSD+CC architecture, with a fixed hidden layer size of 64. Average of 5 runs for each setup.

Furthermore, our results suggest the existence of an optimal range for both the hidden layer size and the clustering loss γ . Beyond this range, further increases do not consistently yield performance improvements. Therefore, determining the optimal balance between these parameters remains a challenge. Figures 4 and 5 show the variation in NMI scores as γ and the hidden layer size are adjusted for the DGCSD + CC variant on the Cora and Citeseer datasets, respectively. These findings underscore the importance of continued exploration of the relationship between network capacity, clustering loss, and overall algorithm performance to identify the most effective configuration for the DGCSD framework.



(a) DGCSO + CC on Cora: Hidden layer size variation.



(b) DGCSO + CC on Citeseer: Hidden layer size variation.

Fig. 5. Variation in the Hidden Layer size for the DGCSO+CC architecture, with a fixed clustering loss γ of 10. Average of 5 runs for each setup.

5 Conclusion

In this paper, we introduce **DGCSO**, an unsupervised deep clustering algorithm that combines seed detection with a GNN-based embedding framework in an end-to-end manner. Our approach leverages both the graph structure and node content information to produce embeddings tailored for clustering tasks. Specifically, we identify a small subset of representative “seed” nodes whose confident cluster assignments guide the overall learning process, thereby providing self-supervision in an otherwise unsupervised setting.

By jointly optimizing a clustering loss along with an appropriate reconstruction or embedding objective, DGCSO learns node representations that naturally separate into cohesive clusters. This design preserves the underlying graph topology and amplifies the signal from the seeds, leading to improved clustering performance. Experiments confirm the effectiveness of our algorithm on benchmark datasets, positioning DGCSO competitively among state-of-the-art clustering approaches.

Another important contribution of this study is the integration of seed detection strategies into the GNN architecture. This approach not only demonstrates the feasibility of combining these two concepts but also lays the groundwork for future research in this direction. By bridging classical graph analysis techniques with deep learning models, this work offers a novel perspective that may serve as a theoretical reference for subsequent studies aiming to enhance graph-based clustering and representation learning.

In future work, we plan to extend our framework to accommodate larger real-world graphs with richer and more diverse feature sets. We also aim to explore adaptive seed detection strategies, more sophisticated embedding architectures, and advanced training schemes to further enhance the scalability and accuracy of DGCSO in diverse application domains.

Acknowledgments. This research was funded by the Coordination for the Improvement of Higher Education Personnel (CAPES) – Code 001, through the provision of a master’s scholarship, and by the São Paulo Research Foundation (FAPESP), grant 20/09835-1 (Alneu Lopes), grant number 22/03090-0, and by CNPq - National Council for Scientific and Technological Development, grant number 420025/2023-5, grant PQ (Alneu Lopes) number 303588/2022-5.

References

1. Bo, D., Wang, X., Shi, C., Zhu, M., Lu, E., Cui, P.: Structural deep clustering network. In: Proceedings of The Web Conference 2020, pp. 1400–1410. WWW ’20, ACM (2020). <https://doi.org/10.1145/3366423.3380214>
2. Cai, H., Zheng, V.W., Chang, K.C.C.: A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Trans. Knowl. Data Eng.* **30**(9), 1616–1637 (2018). <https://doi.org/10.1109/TKDE.2018.2807452>
3. Cheng, J., He, C., Han, K., Chen, G., Liang, W., Tang, Y.: Unveiling community structures in static networks through graph variational bayes with evolution information. *Neurocomputing* **576**, 127349 (2024). <https://doi.org/10.1016/j.neucom.2024.127349>
4. Choong, J.J., Liu, X., Murata, T.: Optimizing variational graph autoencoder for community detection with dual optimization. *Entropy* **22**(2), 197 (2020). <https://doi.org/10.3390/e22020197>
5. Fortunato, S., Barthélemy, M.: Resolution limit in community detection. *Proc. Nat. Acad. Sci.* **104**(1), 36–41 (2007). <https://doi.org/10.1073/pnas.0605965104>
6. Freeman, L.C.: A set of measures of centrality based on betweenness. *Social Networks* **1**(1), 215–239 (1977)
7. Freeman, L.C.: Centrality in social networks: Conceptual clarification. *Social Networks* **1**(3), 215–239 (1978)
8. Guo, L., Dai, Q.: End-to-end variational graph clustering with local structural preservation. *Neural Comput. Appl.* **34**(5), 3767–3782 (2021). <https://doi.org/10.1007/s00521-021-06639-7>
9. Jiao, Z., Li, X.: An end-to-end deep graph clustering via online mutual learning. *IEEE Transactions on Neural Networks and Learning Systems* **36**(2), 3847–3854 (2025). <https://doi.org/10.1109/TNNLS.2024.3353217>
10. Jin, D., Li, Z., Yang, L., He, D., Jiao, P., Zhai, L.: Adversarial capsule learning for network embedding. In: 2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI), pp. 214–221 (2019). <https://doi.org/10.1109/ICTAI.2019.00038>
11. Kempe, D., Kleinberg, J., Tardos, É.: Maximizing the spread of influence through a social network. In: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 137–146. ACM (2003)
12. Kipf, T.N., Welling, M.: Variational graph auto-encoders. In: Proceedings of the International Conference on Learning Representations (Workshop Track), arXiv preprint [arXiv:1611.07308](https://arxiv.org/abs/1611.07308) (2016)
13. Kullback, S., Leibler, R.A.: On information and sufficiency. *Ann. Math. Stat.* **22**(1), 79–86 (1951)
14. Li, Z., Zhu, C.: Self-supervised graph clustering via attention auto-encoder with distribution specificity. *Multimedia Syst.* **30**(3), 150 (2024). <https://doi.org/10.1007/s00530-024-01346-4>

15. Moradan, A., Draganov, A., Mottin, D., Assent, I.: Ucode: unified community detection with graph convolutional networks. *Mach. Learn.* **112**(12), 5057–5080 (2023). <https://doi.org/10.1007/s10994-023-06402-0>
16. Pan, S., Hu, R., Fung, S.F., Long, G., Jiang, J., Zhang, C.: Learning graph embedding with adversarial training methods. *IEEE Trans. Cybern. Cybern.* **50**(6), 2475–2487 (2020). <https://doi.org/10.1109/tcyb.2019.2932096>
17. Sabidussi, G.: The centrality index of a graph. *Psychometrika* **31**(4), 581–603 (1966)
18. Salha, G., Hennequin, R., Vazirgiannis, M.: Simple and Effective Graph Autoencoders with One-Hop Linear Models, pp. 319–334. Springer International Publishing (2021). https://doi.org/10.1007/978-3-030-67658-2_19, http://dx.doi.org/10.1007/978-3-030-67658-2_19
19. Shen, Z., Guo, X., Feng, B., Cheng, H., Ni, S., Dong, H.: Adversarial learning based residual variational graph normalized autoencoder for network representation. *Inf. Sci.* **640**, 119055 (2023). <https://doi.org/10.1016/j.ins.2023.119055>
20. Sheng, J., Yang, Z., Wang, B., Chen, Y.: Attribute graph embedding based on multi-order adjacency views and attention mechanisms. *Mathematics* **12**(5), 697 (2024). <https://doi.org/10.3390/math12050697>
21. Su, X., Xue, S., Liu, F., Wu, J., Yang, J., Zhou, C., Hu, W., Paris, C., Nepal, S., Jin, D., Sheng, Q.Z., Yu, P.S.: A comprehensive survey on community detection with deep learning. *IEEE Transactions on Neural Networks and Learning Systems* **35**(4), 4682–4702 (2024). <https://doi.org/10.1109/TNNLS.2021.3137396>
22. Sun, D., Liu, L., Luo, B., Ding, Z.: Glass: a graph Laplacian autoencoder with subspace clustering regularization for graph clustering. *Cogn. Comput.* **15**(3), 803–821 (2023). <https://doi.org/10.1007/s12559-022-10098-0>
23. Wang, C., Pan, S., Yu, C.P., Hu, R., Long, G., Zhang, C.: Deep neighbor-aware embedding for node clustering in attributed graphs. *Pattern Recogn.* **122**, 108230 (2022). <https://doi.org/10.1016/j.patcog.2021.108230>
24. Wang, Y., Chang, D., Fu, Z., Zhao, Y.: Seeing all from a few: Nodes selection using graph pooling for graph clustering. *IEEE Trans. Neural Netw. Learn. Syst.* **35**(5), 7231–7237 (2024). <https://doi.org/10.1109/tnnls.2022.3210370>
25. Wu, D., Guo, X., Luo, X., Qiao, Z., Ma, J.: Adaptive harmony learning and optimization for attributed graph clustering. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2022). <https://doi.org/10.1109/IJCNN55064.2022.9892820>
26. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* **32**(1), 4–24 (2021). <https://doi.org/10.1109/TNNLS.2020.2978386>
27. Yang, Y., Han, B., Ran, Z., Gao, M., Wei, Y.: Attributed graph embedding with random walk regularization and centrality-based attention. *Mathematics* **11**(8), 1830 (2023). <https://doi.org/10.3390/math11081830>
28. Zhang, W., Shang, R., Li, Z., Sun, R., Du, J.: Personalized web page ranking based graph convolutional network for community detection in attribute networks. *IEEE Access* **11**, 84270–84282 (2023). <https://doi.org/10.1109/ACCESS.2023.3303210>